

# Multi-Agent CAD Code Generation (Supplementary Materials)

This appendix provides additional details supporting the main paper, organized as follows:

- A. **CAD AST Representation:** Description of the CAD AST representation with illustrative examples.
- B. **Multimodal Dataset Preparation:** Details on dataset construction and preprocessing.
- C. **Implementation Details:** Implementation details of CAD-Factory and baseline methods.
- D. **Additional Experiments:** Additional quantitative and qualitative results.
- E. **Failure Cases and Discussion:** Analysis of representative failure cases and additional discussions.

## A CAD AST Representation

Our CAD Abstract Syntax Tree (AST) representation is a purely hierarchical structure that encodes modeling organization and operation dependencies without numerical parameters. Figure 1 provides an illustrative example. By augmenting the AST with geometric parameters, it can be directly extended into an executable CAD program compatible with mainstream CAD kernels such as PythonOCC [Riegel et al. 2016].

### A.1 AST Hierarchy

Following the real-world CAD modeling process, each model consists of one or more parts (outer hierarchy), with each part constructed through a fixed sequence: coordinate system, sketch profile, and extrusion operation (inner hierarchy). Within each sketch profile, the geometry follows a nested structure of *face*  $\rightarrow$  *loop*  $\rightarrow$  *curve*. Each sketch comprises one or more faces, each face is bounded by one or more closed loops, and each loop consists of multiple curve primitives, including line, arc, and circle.

### A.2 Token Representation

To support this hierarchical design, we introduce 21 specialized node tokens, as listed in Table 1 of the main paper. We provide their definitions below.

**Special Tokens.** Each CAD model begins with `<|SOC|>` and ends with `<|EOC|>`, with `<|part|>` indicating the start of each part.

**Coordinate System Tokens.** A coordinate system starts with `<|coord|>`, followed by `<|euler|>` (Euler angles) and `<|trans|>` (translation vector) to specify the sketch plane orientation and position.

**Sketch Profile Tokens.** A sketch profile begins with `<|sketch|>`, containing one or more `<|face|>`, where each face consists of one or more `<|loop|>` and each loop is composed of curve primitives `<|line|>`, `<|arc|>`, or `<|circle|>`. In the CAD program, these primitives are instantiated with continuous geometric parameters to specify their shapes in the sketch plane:

- `<|line|>`: Start and End points.
- `<|arc|>`: Start, Mid, and End points.
- `<|circle|>`: Center point and Radius.

**Extrusion Operation Tokens.** An extrusion block begins with `<|extrusion|>`, specifying extrusion distances using `<|e_t|>` and

`<|e_o|>`, corresponding to directions toward and opposite the sketch plane normal. The overall sketch scale is controlled by `<|size|>`, and `<|operation|>` defines how the extruded body interacts with existing geometry through the following Boolean operations:

- `<|New|>`: Create a new solid body.
- `<|Join|>`: Union with existing geometry.
- `<|Cut|>`: Subtraction from existing geometry.
- `<|Inter|>`: Intersection with existing geometry.

### A.3 Parameter Quantization

For each CAD program, following prior work [Wu et al. 2021], we apply 8-bit uniform quantization to all continuous parameters (e.g., coordinates and extrusion distances), discretizing them into integers in the range  $[0, 255]$ . This discretization supports stable numerical encoding and efficient learning of geometric parameters.

## B Multimodal Dataset Preparation

This section provides implementation details of image rendering and MLLM annotation for our multimodal dataset.

### B.1 CAD Rendering.

As described in the main paper, all rendered views are generated from the 110K cleaned CAD models filtered from DeepCAD [Wu et al. 2021]. Specifically, each model is decomposed into part-wise CAD programs, converted to STEP format, and rendered using the PythonOCC kernel [Riegel et al. 2016]. In the following, we detail how semantic encoding, the modeling process, and geometric cues are conveyed through carefully designed rendering strategies.

**Semantic Encoding.** To visually indicate modeling semantics, each Boolean operation is associated with a unique color applied to its resulting solid: red for New, green for Join, blue for Cut, and yellow for Inter. This color coding helps the MLLM distinguish construction steps and understand modeling semantics, with all rendered views in the annotation pipeline following this scheme to ensure consistency.

**Modeling Process.** One key challenge in MLLM-based annotation is enabling the model to understand the step-by-step design logic of CAD modeling. To address this, we develop a *CAD Assembly Renderer* to generate assembly operation views that highlight each part’s role in the construction process. Specifically, for each CAD model, we construct STEP models for individual parts and reassemble them according to their original spatial configuration. During rendering, only the active part is shown in its predefined color, while the remaining parts are displayed in translucent gray to convey the full assembly layout. These views are rendered from four isometric angles (45°, 135°, 225°, and 315°) to ensure diverse spatial perspectives.

**Geometric Cues.** To further support MLLM annotation with fine-grained geometric information, we render two additional views for each part: sketch profiles and extruded geometries. Sketch profiles are rendered from a top-down orthographic view to illustrate the 2D curve layout and connectivity. Extruded geometries are rendered

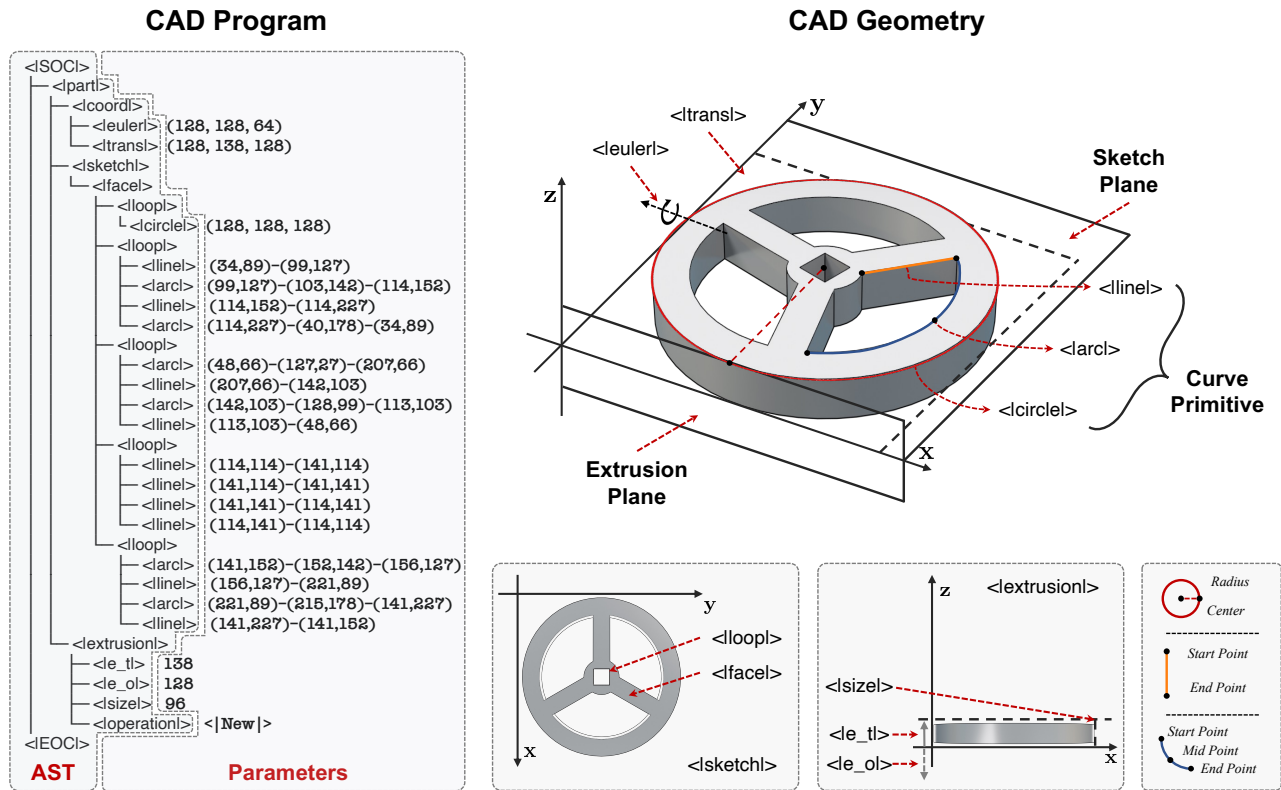


Fig. 1. Hierarchical CAD AST representation illustrating part-wise decomposition into coordinate systems, sketch profiles, and extrusion operations.

from the same four isometric angles to reveal the resulting 3D solid shape.

## B.2 Parameter-Partial Text-CAD Annotation

In addition to the parameter-free setting used in the main paper, we construct a parameter-partial annotation variant by introducing a limited set of intuitive parameter cues (e.g., extrusion distance), while still excluding low-level sketch specifications. This setting bridges abstract design intent and human-interpretable parametric constraints. In this variant, the decomposed CAD code  $\hat{C}^i$  replaces the AST representation  $\mathcal{A}^i$  as input to the MLLM, together with a modified prompt  $\mathcal{P}_{\text{partial}}$ , as illustrated in Figure 2, yielding:

$$\mathcal{T}_p^i = \text{MLLM}(\mathcal{I}_o^i, \mathcal{I}_e^i, \mathcal{I}_s^i, \hat{\mathcal{C}}^i; \mathcal{P}_{\text{partial}}) \text{ for } i = 1, \dots, n. \quad (1)$$

As in the parameter-free setting, part-level outputs are sequentially aggregated and further processed by the MLLM, conditioned on the complete assembly rendering  $\mathcal{I}$ , to produce a global textual description, which, together with the part-level descriptions, forms the final parameter-partial annotation.

### B.3 Annotation Details

**Prompting Methods.** As described in the main paper, we define two annotation settings: parameter-free and parameter-partial. In the parameter-free setting, each prompt is conditioned on a triplet of rendered images ( $I_a^i, I_e^i, I_s^i$ ) and the corresponding CAD AST  $\mathcal{A}^i$

without any numeric parameters. In the parameter-partial setting, the prompt uses the same image triplet but replaces the AST with a partially specified CAD code  $\hat{C}^i$  that includes only coordinate system and extrusion parameters. Both settings share the same prompt template:

**Base Prompt:** *[system] You are a professional CAD annotation assistant. Your task is to write a clear and factual description of the given CAD part. [Inputs] One rendering of the sketch used to create this part. One rendering of the solid showing the extracted sketch. One rendering showing the complete CAD model with the current operation highlighted. One AST showing the structural tokens for this part. (Parameter-partial Setting: One CAD code showing the structural tokens with partial parameters for this part.) [Task] Describe the main outline shape first, then mention detailed geometry. Also state the relative position of the part within the full model. Use clear and concise language to describe the geometry and spatial placement, aligned with the highlighted modeling step. [Constraints] Focus only on the highlighted area (red = create, green = join, blue = cut, yellow = intersect; other parts are transparent gray).*

We extend the base prompt with an additional constraint to define the parameter-free  $\mathcal{P}_{\text{free}}$  and parameter-partial  $\mathcal{P}_{\text{partial}}$  prompts as follows:

- $\mathcal{P}_{\text{free}}$ : Do not include any parameters or freely infer any missing parameters in the description.

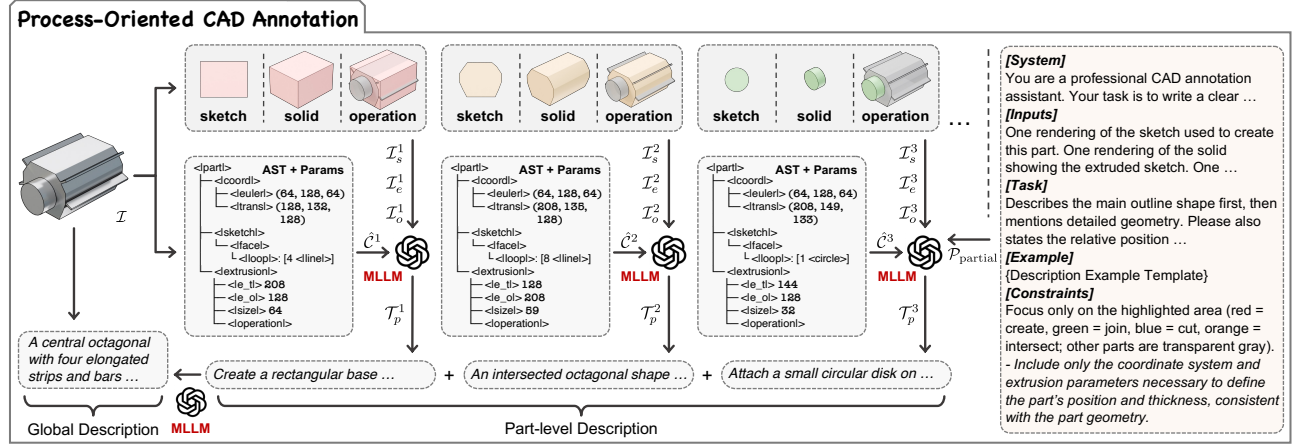


Fig. 2. **Process-Oriented Annotation Pipeline (Parameter-Partial Setting).** The pipeline follows the same process-oriented design as the parameter-free setting, but adopts a modified prompt  $\mathcal{P}_{\text{partial}}$  and decomposed CAD code  $\hat{C}^i$  to enable part-level descriptions with limited parameter cues.

- $\mathcal{P}_{\text{partial}}$ : Include only the coordinate system and extrusion parameters necessary to define the part's position and thickness, consistent with the part geometry.

These minimal prompt adjustments enable consistent formatting while accommodating different levels of semantic specificity.

**Textual Description Construction.** After annotation, part-level textual descriptions are sequentially aggregated in modeling order to reflect the procedural nature of CAD construction, rather than relying on one-step captioning of the final model. Based on the aggregated descriptions together with the complete assembly rendering  $I$ , a concise global description is further generated and prepended, forming the final textual description that captures both the overall design intent and the step-by-step modeling process. The global description is generated using the following prompt:

**[System]** You are a professional CAD annotation assistant. Your task is to summarize the overall design intent and structure of a CAD model.  
**[Inputs]** Aggregated part-level textual descriptions in modeling order. One rendering showing the complete assembled CAD model.  
**[Task]** Provide a concise global description of the CAD model, highlighting its overall shape, major components, and structural organization. Use clear and factual language.

**Annotation Quality Control.** To reduce hallucinations and improve alignment with executable CAD programs, we apply automated consistency checks during dataset construction. In particular, the number of generated modeling-step descriptions must match the number of construction steps in the corresponding CAD program. Samples with semantic inconsistencies, missing components, or mismatches between textual descriptions and highlighted modeling stages are further manually inspected and revised by experienced CAD users. This hybrid verification strategy ensures that the generated descriptions remain faithful to both local geometric evidence and the sequential modeling process.

**Editing Data Preparation.** As a representative downstream application of language-driven CAD modeling, we construct an editing instruction dataset based on CAD models produced during the preceding text-to-CAD generation stage. Each editing sample is

formulated as a triplet  $(C, C', \mathcal{E}_{\text{edit}})$ , where  $C$  and  $C'$  denote the original and edited CAD codes, respectively, and  $\mathcal{E}_{\text{edit}}$  specifies the corresponding editing intent. Notably, editing operations are not inherently feasible across arbitrary parts or modeling steps. For instance, when interdependencies exist among components within a CAD model, indiscriminate removal or modification of certain parts or primitives may lead to invalid geometries or ill-formed modeling semantics. Therefore, we explicitly constrain and verify editing feasibility, retaining only editing pairs that are both structurally and semantically valid.

Based on these requirements, we construct the editing dataset around three canonical language-driven CAD editing scenarios: "Addition", "Removal", and "Modification". Specifically, *Addition* corresponds to augmenting an existing CAD model with newly introduced geometric parts; *Removal* involves deleting specified local geometric parts or feature structures; and *Modification* refers to adjusting local geometric shapes or key parameters, such as dimensions, positions, or scales. The dataset construction procedures and feasibility validation criteria for each scenario are detailed below.

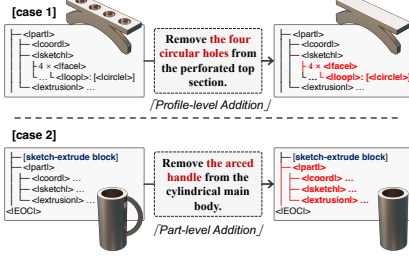
For removal-based editing ("Removal"), independently formable geometric parts are treated as basic editing units. Candidate removable objects are automatically identified from the existing CAD code using an LLM, including solid components introduced by modeling operations as well as feature structures with well-defined local semantics (e.g., closed profiles). To provide sufficient global context, we additionally supply a parameter-free modeling text description of the original CAD model, enabling a holistic understanding of the overall structure. Each candidate is subsequently subjected to feasibility checks. A removal operation is retained only if the resulting CAD model remains executable, geometrically valid, and semantically consistent. The final removal sample consists of paired original and edited CAD code, together with LLM-generated editing instructions that explicitly specify the removed part and its role within the model. The prompting strategy and corresponding removal cases are illustrated on the left side of Figure 3.

## Removal-based Editing

**[System Task]**  
You are a CAD code editing assistant. You are given a CAD code that represents a CAD model, together with its modeling description. Your task is to generate a single removal-based CAD editing sample, including:  
1. One realistic natural-language editing instruction.  
2. The corresponding edited CAD code after removal.

**[Constraints]**  
— Only remove one or more formable geometric parts or local features.  
— Do not remove the primary structural body of the model.  
— The resulting CAD code must remain executable, geometrically valid, and semantically coherent.  
— The editing instruction must be written in natural, human-like language.  
— Specify the removed part and its semantic role in the model.  
— Do not mention CAD tokens, parameters, coordinates, or implementation details in the instruction.

**[Example]**  
Input: {Original CAD Code + Original Text Description Example}  
Output: {Editing Instruction + Edited CAD Code Example}

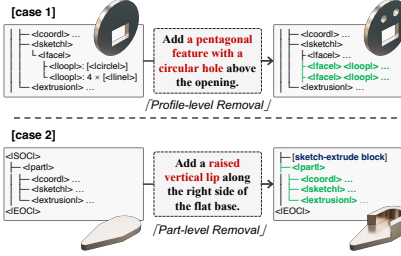


## Addition-based Editing

**[System Task]**  
You are a CAD code editing assistant. You are given a CAD code that represents a 3D CAD model, together with its modeling description. Your task is to generate a single addition-based CAD editing sample, including:  
1. One realistic natural-language editing instruction.  
2. The corresponding edited CAD code after addition.

**[Constraints]**  
— Add one or more geometric parts or local features to the existing model.  
— The added geometry should be compatible with the existing design.  
— The resulting CAD code must remain executable, geometrically valid, and semantically coherent.  
— The editing instruction must be written in natural, human-like language.  
— Describe the added part and its intended semantic role in the model.  
— Do not mention CAD tokens, parameters, coordinates, or implementation details in the instruction.

**[Example]**  
Input: {Original CAD Code + Original Text Description Example}  
Output: {Editing Instruction + Edited CAD Code Example}



## Modification-based Editing

**[System Task]**  
You are a CAD code editing assistant. You are given a CAD code that represents a 3D CAD model, together with its modeling description. Your task is to generate a single modification-based CAD editing sample, including:  
1. One realistic natural-language editing instruction.  
2. The corresponding edited CAD code after modification.

**[Constraints]**  
— Do not add or remove any geometric parts.  
— Modify existing geometry only by adjusting valid attributes.  
— Preserve the overall structure and intended function of the model.  
— The resulting CAD code must remain executable, geometrically valid, and semantically coherent.  
— The editing instruction must be written in natural, human-like language.  
— Specify the target part or feature and the intended modification.

**[Example]**  
Input: {Original CAD Code + Original Text Description Example}  
Output: {Editing Instruction + Edited CAD Code Example}

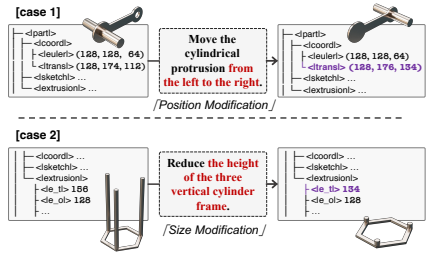


Fig. 3. Prompt design and representative examples for editing dataset construction.

For addition-based editing (“Addition”), samples are constructed by introducing new geometric components into an existing CAD model. Specifically, we start from a valid base CAD code and augment it by inserting one or more additional geometric parts or local features. The overall pipeline follows the same procedure as removal-based editing: we provide the original CAD code together with its modeling description, and then leverage an LLM to generate the corresponding editing instruction and the updated CAD code. This instruction describes the addition of the specified component and clarifies necessary spatial or assembly constraints, such as relative position, alignment, or connectivity. Representative addition cases are illustrated in the middle of Figure 3. Notably, removal and addition operations naturally form inverse transformations at the geometric level. As a result, valid addition samples can also be constructed by reinserting previously removed parts into edited CAD models, thereby restoring the original code.

For modification-based editing (“Modification”), we focus on adjusting the shape or key parameters of existing geometric parts. Specifically, we use an LLM to select semantically meaningful and commonly edited parameters based on the original CAD code and modeling description, such as extrusion distance, size ratio, or local translation and scaling. These parameters are perturbed within predefined valid ranges. For each candidate modification, feasibility checks ensure that (i) the resulting CAD code remains executable and syntactically valid, and (ii) the modification aligns with the intended editing semantics. Correspondingly, the editing instruction explicitly specifies the target part, the parameter to be modified, and the direction of change (e.g., increase or decrease), thereby simulating typical parametric editing behaviors in practical CAD design. We provide detailed prompt and example cases on the right side of Figure 3.

For all three types of editing samples, we apply executability and consistency checks to ensure that both pre-edit and post-edit CAD

Table 1. Token cost for Text-CAD dataset annotation and editing dataset construction.

| Task                         | Total Tokens (million) |
|------------------------------|------------------------|
| Parameter-Free Annotation    | 160M                   |
| Parameter-Partial Annotation | 170M                   |
| Editing Dataset Annotation   | 30M                    |

programs can be correctly executed to produce valid 3D geometry while preserving reasonable structural and semantic correspondence. Throughout the data construction process, GPT-4o is used to generate editing instructions.

**Token Cost.** To ensure high-quality text generation, all annotation tasks are conducted using GPT-4o, which provides robust multi-modal reasoning over CAD renderings and structural inputs. As summarized in Table 1, the parameter-free annotation setting consumes approximately 160M tokens, while the parameter-partial setting requires around 170M tokens due to the inclusion of numerical parameters. In addition, constructing the CAD editing dataset incurs an extra cost of approximately 30M tokens. All annotation processes are executed on a single NVIDIA A100 GPU, with the entire pipeline completed in approximately five days.

**Annotation LLM Discussion.** We investigate the impact of different MLLMs for annotating textual descriptions during dataset construction. In our experiments, we find that general-purpose open-source MLLMs (e.g., Qwen-VL [Bai et al. 2023]) are also sufficient for producing accurate and semantically aligned annotations, making them a cost-effective choice for large-scale data annotation. In contrast, GPT-4o provides richer linguistic expressions and greater diversity in annotation styles, which helps mitigate potential overfitting to repetitive descriptions during training. In practice, increased annotation diversity improves the robustness and generalization of language-driven CAD generation models.



**Text-to-AST Generation****[System]**

You are a CAD code generation assistant. Your task is to convert a natural-language description into a structured CAD token-based abstract syntax tree (AST).

**[Task]**

Generate a token-only CAD AST from a natural-language description that strictly follows the predefined structure and token rules. Each CAD model consists of one or more `<|part|>` blocks, and each `<|part|>` must follow this sequence:

1. `<|coord|>` Block  
Defines the spatial pose of the part, containing exactly one `<|euler|>` for Euler rotation angles and one `<|trans|>` for translation.
2. `<|sketch|>` Block  
Defines the 2D sketch profile of the part, containing up to ten `<|face|>` blocks. Each `<|face|>` contains one or more `<|loop|>` elements.  
A `<|loop|>` contains either a single `<|circle|>` or a combination of `<|line|>` and `<|arc|>`.  
— `<|line|>` → Defines a straight line  
— `<|arc|>` → Defines a curved segment  
— `<|circle|>` → Defines a complete circle
3. `<|extrusion|>` Block  
Encodes the 3D operation that transforms the sketch into a solid. Must contain `<|e_t|>`, `<|e_o|>`, `<|size|>`, and `<|operation|>`.

**[Constraints]**

The output must contain only CAD tokens and must begin with `<|SOC|>` and end with `<|EOC|>`. No numeric values, coordinates, angles, distances, or Boolean operation types are allowed.

**[Example]**

Input: (CAD Text Description Example)  
Output: (Corresponding CAD AST Example)

**Text+AST-to-Code Completion****[System]**

You are a CAD code generation assistant. Your role is to generate a fully parameterized CAD code by filling in numeric values and operation types for each placeholder token in a given abstract syntax tree (AST) structure.

**[Task]**

Given a natural-language text description of a CAD model and its corresponding AST representation, produce a valid, parameter-complete CAD code, strictly following the parameterization rules:

1. `<|coord|>` Block  
— `<|euler|>` (x, y, z) → Euler rotation angles in degrees  
— `<|trans|>` (x, y, z) → Translation in model units  
Default value (128, 128, 128) indicates no rotation and origin-centered placement.
2. `<|sketch|>` Block  
— `<|line|>` (start\_x, start\_y)-(end\_x, end\_y) → A straight line from start to end points  
— `<|arc|>` (start\_x, start\_y)-(mid\_x, mid\_y)-(end\_x, end\_y) → A curved segment defined by start, middle, and end points  
— `<|circle|>` (center\_x, center\_y, radius) → A full circle defined by center and radius
3. `<|extrusion|>` Block  
— `<|e_t|>` → Extrusion depth toward the sketch normal direction  
— `<|e_o|>` → Extrusion depth opposite to the normal direction  
— `<|size|>` → A scaling factor for sketch size  
— `<|operation|>` → One of the valid modeling operations: `<|New|>`, `<|Join|>`, `<|Cut|>`, or `<|Inter|>`

The default `<|e_t|>` 128 and `<|e_o|>` 128 indicate zero extrusion and must be inferred from the input text.

**[Constraints]**

All integer parameters are quantized to the range [0, 255], and each `<|operation|>` must be one of the four symbolic values: `<|New|>`, `<|Join|>`, `<|Cut|>`, or `<|Inter|>`. No primitives may be invented or omitted, and no parameters may be assumed unless they are explicitly implied by the description.

The output must strictly match the structure of the provided AST and complete all placeholder tokens with explicit numeric or symbolic parameters derived from the text description.

**[Example]**

Input: (CAD Text Description Example) (Corresponding CAD AST Example)  
Output: (Corresponding CAD Code Example)

**CAD Code Editing****[System]**

You are a CAD code editing assistant. Your task is to apply a user-provided editing instruction to an existing fully parameterized CAD code.

**[Task]**

Given the following inputs:

1. A fully parameterized valid CAD code.
2. A user-specified editing instruction describing the desired modification.
3. A fixed rendering color palette that maps each color to a specific `<|part|>` index for visual identification:  
(red: part1, green: part2, blue: part3, yellow: part4, purple: part5, cyan: part6, orange: part7, magenta: part8, dark green: part9, brown: part10, ...)

The objective is to produce a modified CAD code that preserves the original structure, except for the changes required to fulfill the editing instruction. The correspondence between the editing instruction and the `<|part|>` blocks in the CAD code must be established based on the predefined rendering color palette.

Each `<|part|>` in the output must retain its original hierarchical structure: — `<|coord|>` Block: `<|euler|>` for Euler rotation angles, `<|trans|>` for translation.

— `<|sketch|>` Block: Contains `<|face|>` and `<|loop|>` elements with `<|line|>`, `<|arc|>`, and `<|circle|>` primitives.

— `<|extrusion|>` Block: Contains `<|e_t|>`, `<|e_o|>`, `<|size|>`, and `<|operation|>`.

**[Constraints]**

Preserve all unchanged parts exactly as in the original CAD code. Modify only the parts identified by the mapping process based on the rendering color palette and the editing instruction.

Ensure that all primitives and parameters in the modified parts are valid and consistent with the editing instruction.

**[Example]**

Input: (Original CAD Code Example) (Editing Instruction Example)  
Output: (Updated CAD Code Example)

Fig. 4. Agentic fine-tuning prompts for text-to-AST generation, text+AST-to-code completion, and CAD code editing.

Table 2. Keyword mappings for CAD AST tokens used in embedding initialization.

| Token                            | Keywords                                       | Token                        | Keywords                                          |
|----------------------------------|------------------------------------------------|------------------------------|---------------------------------------------------|
| <code>&lt; SOC &gt;</code>       | start, beginning, CAD, geometry                | <code>&lt; EOC &gt;</code>   | end, completion, CAD, geometry                    |
| <code>&lt; part &gt;</code>      | part, component, element, CAD                  | <code>&lt; coord &gt;</code> | coordinate, position, reference, spatial          |
| <code>&lt; euler &gt;</code>     | rotation, orientation, angle, euler, transform | <code>&lt; trans &gt;</code> | translation, position, offset, displacement, move |
| <code>&lt; sketch &gt;</code>    | sketch, profile, outline, design, drawing      | <code>&lt; face &gt;</code>  | face, surface, plane, region, boundary            |
| <code>&lt; loop &gt;</code>      | loop, contour, boundary, closed, path          | <code>&lt; line &gt;</code>  | line, straight, geometry, primitive               |
| <code>&lt; circle &gt;</code>    | circle, round, geometry, primitive             | <code>&lt; arc &gt;</code>   | arc, circular, geometry, primitive                |
| <code>&lt; extrusion &gt;</code> | extrusion, extend, operation, volume           | <code>&lt; e_t &gt;</code>   | extrude, towards, normal, direction, positive     |
| <code>&lt; e_o &gt;</code>       | extrude, opposite, normal, direction, negative | <code>&lt; size &gt;</code>  | size, scale, dimension, magnitude                 |
| <code>&lt; operation &gt;</code> | geometric, action, process, boolean, operation | <code>&lt; New &gt;</code>   | new, create, base, boolean, operation             |
| <code>&lt; Cut &gt;</code>       | cut, subtract, remove, boolean, operation      | <code>&lt; Join &gt;</code>  | join, merge, combine, boolean, operation          |
| <code>&lt; Inter &gt;</code>     | intersect, union, overlap, boolean, operation  |                              |                                                   |

## C Implementation Details

### C.1 CAD-Factory System

**Agent Initialization.** In our framework, the planning (*[Manager]*), coding (*[Programmer]*), and editing (*[Editor]*) agents are all instantiated from LLaMA-3.2-3B [Meta 2024]. While each agent is fine-tuned on its corresponding instruction dataset for specific subtasks, they share the same LLM backbone and tokenizer enhanced with CAD-specific knowledge.

**Tokenizer Initialization.** As detailed in the main paper, to facilitate effective CAD AST learning, we extend the tokenizer vocabulary with 21 domain-specific tokens that cover structural blocks, sketch primitives, coordinate specifications, and extrusion operations. To ensure semantic grounding, we initialize the embedding of each AST token using the average embedding of its associated descriptive keywords, following Equation 2. The full list of token-keyword mappings is provided in Table 2, where each token is assigned

several keywords that reflect its functional role in the CAD modeling pipeline. All descriptive keywords are automatically generated and refined using GPT-4o based on natural language explanations of token semantics.

**Training Setup Details.** All experiments are conducted using four NVIDIA A100 (80GB) GPUs with bf16 mixed-precision training. During domain-adaptive pretraining (3 epochs), we update only the tokenizer adaptation layer in the first epoch to facilitate adaptation to special AST tokens. Agentic fine-tuning (5 epochs) is conducted using LoRA [Hu et al. 2022], inserted into both attention and feedforward modules. Pretraining, individual fine-tuning, and joint tuning require approximately 32, 18, and 3 hours, respectively.

**Agentic Fine-tuning Details.** In agentic fine-tuning, we define task-specific prompts for the planning (text-to-AST generation), coding (text+AST-to-code completion), and editing (CAD code editing) agents, ensuring that each agent focuses exclusively on its designated task, as illustrated in Figure 4. These prompts explicitly specify

**[System]**

You are a CAD code generation assistant. Your task is to convert a natural-language description into fully parameterized CAD token sequences.

**[Task]**

To enable structured and executable CAD model generation from text, the output must strictly follow a well-defined token format that reflects the procedural modeling process. The generated output must begin with `<ISOC>` and end with `<IEOC>`, containing only CAD tokens and integer (or symbolic) parameters. Each design consists of multiple `<part>` blocks, and each `<part>` must follow a consistent structure in the following order:

1. `<coord>` Block

Defines the spatial pose of the part using:

- `<leuler>` (x, y, z) → Euler rotation angles in degrees
- `<ltransl>` (x, y, z) → Translation in model units

Default value (128, 128, 128) indicates no rotation and origin-centered placement.

2. `<sketch>` Block

Defines the 2D sketch profile of the part, composed of multiple `<lfacel>` blocks. Each `<lfacel>` contains one or more `<loop>` elements. A loop must either consist of a single `<circle>`, or a combination of `<llinel>` and `<larc>` tokens, and the supported sketch primitives include:

- `<llinel>` (start\_x, start\_y)-(end\_x, end\_y) → A straight line from start to end points
- `<larc>` (start\_x, start\_y)-(mid\_x, mid\_y)-(end\_x, end\_y) → A curved segment defined by start, middle, and end points
- `<circle>` (center\_x, center\_y, radius) → A full circle defined by center and radius

3. `<extrusion>` Block

Encodes the 3D modeling operation that transforms the sketch into volume. It must contain exactly:

- `<le_tl>` → Extrusion depth toward the sketch normal direction
- `<le_ol>` → Extrusion depth opposite to the sketch normal direction
- `<lsizel>` → A scaling factor for sketch size
- `<loperation>` → One of the valid modeling operations: `<lNewl>`, `<lJoinl>`, `<lCutl>`, or `<lInterl>`

The default reference values are `<le_tl>` 128, `<le_ol>` 128, and `<lsizel>` 128. Final values must be inferred from the input text.

**[Constraints]**

All geometry, operations, and parameters must be faithfully inferred from the natural-language description.

The token sequence must strictly follow the modeling description order implied in the text.

All integer parameters are quantized to the range [0, 255], and each `<loperation>` must be one of the four symbolic values: `<lNewl>`, `<lJoinl>`, `<lCutl>`, or `<lInterl>`.

**[Example]**

Input: {CAD Text Description Example}

Output: {Corresponding CAD Code Example}

Fig. 5. Prompt design for LLM-based baselines in text-to-CAD code generation tasks.

the target task, input-output format, and expected constraints, enabling each agent to learn domain-relevant reasoning patterns while maintaining clear separation between tasks.

## C.2 Text-to-CAD Baselines

For text-to-CAD generation, we evaluate a diverse set of baselines spanning general-purpose LLMs, coding-oriented LLMs, multi-modal methods, and task-specific approaches. Considering that some recent methods do not have publicly available implementations, we include representative methods from each category and reproduce them following the original papers. Together, these baselines provide broad coverage of existing text-to-CAD paradigms, enabling a comprehensive and systematic comparison with our system. Methods focusing on B-rep generation are not included as baselines due to their different output space and evaluation protocol, making direct comparison less appropriate. Implementation details for each method are provided below.

**General-purpose and Coding-oriented LLMs.** General-purpose LLMs (GPT-4 [Achiam et al. 2024], Gemini-1.5 [Team et al. 2024], DeepSeek-V3 [Liu et al. 2025], and Qwen-3-14B [Yang et al. 2025]) and the coding-oriented LLM (Claude-4) are implemented under a unified inference protocol. To enable a fair and comprehensive comparison across general-purpose LLMs, we follow prior works and adopt stable and commonly used API versions, rather than emphasizing the advantage of any particular API release.

All LLM-based methods adopt a single-stage generation paradigm, directly producing executable CAD programs from input text

descriptions. We follow a standard LLM application strategy by providing rule-based prompts and modeling examples during inference. To improve generation reliability, we allow up to two rounds of routed feedback and automatic code refinement to mitigate failures caused by syntactic errors. The prompting strategy used for these models is illustrated in Figure 5.

**Multi-modal Methods.** For multi-modal methods, the core idea is to incorporate visual cues as additional feedback signals for MLLM-based CAD code generation.

For 3D-PreMise [Yuan et al. 2024], since no official implementation is publicly available, we reproduce the method following the best-performing one-shot setting reported in the original paper. Specifically, we first generate an initial CAD program from the input text description, then export the resulting mesh and render it from four top-down viewpoints at azimuth angles of 0°, 90°, 180°, and 270°. The generated CAD code and rendered images are subsequently fed back into the MLLM for visual self-verification and feedback-based refinement to obtain the final model. All prompting strategies follow the configurations described in the original paper. For cases where syntactic errors lead to failures, we allow up to two rounds of automatic code correction. GPT-4o is used as the base MLLM for both generation and refinement.

For CADVerify [Alrashedy et al. 2025] (also referred to as CAD-CodeVerify), we reproduce the method following the procedures described in the original paper. CADVerify consists of three main stages. First, an initial CAD program is generated from the input textual description. To ensure executability, we allow up to two rounds of code repair to resolve syntax or execution errors. Second,

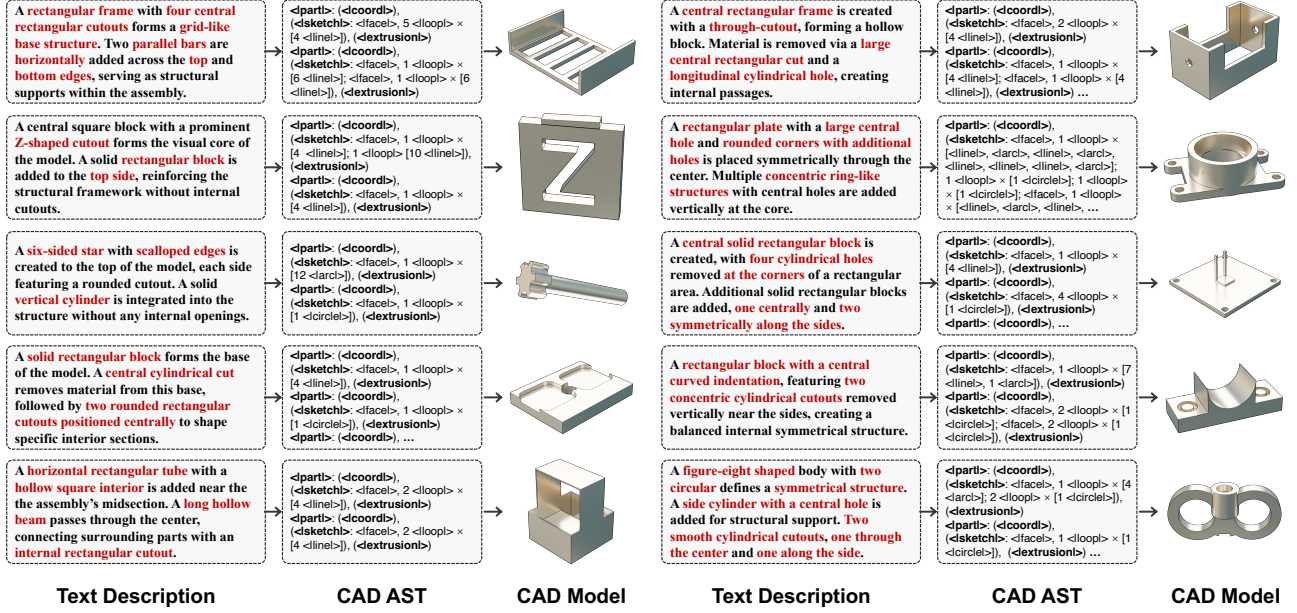


Fig. 6. Additional qualitative results of parameter-free text-to-CAD generation. Given a text description, CAD-Factory first infers an explicit AST representation capturing the underlying model structure, which is then used to produce accurate CAD code and the final 3D model.

we generate a set of binary verification questions (five in our experiments) from the input text to decompose modeling requirements into verifiable geometric and structural constraints. The generated CAD program is executed to produce a mesh, which is rendered from four viewpoints at azimuth angles of 0°, 90°, 180°, and 270°. These images are fed to the MLLM for visual question answering, producing "Yes" / "No" / "Unclear" judgments. Only failed cases are aggregated to generate executable-level feedback that identifies geometric inconsistencies or parameter misconfigurations, covering structure, quantity, symmetry, and relative spatial relationships as defined in the original paper. Finally, the aggregated feedback is used as additional conditioning to guide targeted refinement of the CAD code. The prompting strategies at each stage are adopted from the original paper. Both the verification and refinement stages are repeated for two rounds. GPT-4o serves as the base MLLM throughout.

**Task-specific Methods.** For task-specific baselines, we include Text2CAD [Khan et al. 2024] and CADFusion [Wang et al. 2025], both explicitly designed for text-to-CAD generation with specialized architectures and training objectives. For each method, we adopt the official open-source implementation together with the released pretrained weights. To reduce distribution mismatch and ensure fair comparison under our annotation setting, we further fine-tune these methods on our dataset while carefully following their original configurations.

### C.3 Unconditional CAD Generation Baselines

For unconditional CAD code generation, we compare our method with several baselines including DeepCAD [Wu et al. 2021], Sketch-Gen [Para et al. 2021], and HNC-CAD [Xu et al. 2023]. We use their

official implementations and follow the original training and evaluation protocols. For evaluation, following prior works, we generate 9,000 samples for each method, with each sample containing 2,000 points, and compare them against 3,000 models randomly selected from the test set.

### C.4 Evaluation Metrics

Following prior works [Khan et al. 2024; Li et al. 2025], we adopt a comprehensive set of metrics for performance evaluation, including F1-score, Median Chamfer Distance (MCD), Minimum Matching Distance (MMD), and Jensen–Shannon Divergence (JSD), to evaluate both parametric accuracy and geometric consistency. We additionally introduce AST accuracy ( $ACC_{ast}$ ) to explicitly evaluate the structural correctness of the generated CAD code. Specifically, F1 measures token-level consistency of sketch and extrusion parameters, MCD quantifies geometric fidelity, MMD and JSD evaluate sample-level and distribution-level similarity between generated and real models, and  $ACC_{ast}$  computes the token-level accuracy of the abstract syntax tree in the predicted CAD program. Formally, AST accuracy is defined as

$$ACC_{ast} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\mathcal{T}_i^{\text{pred}} = \mathcal{T}_i^{\text{gt}}), \quad (2)$$

where  $N$  denotes the total number of AST tokens, and  $\mathcal{T}_i^{\text{pred}}$  and  $\mathcal{T}_i^{\text{gt}}$  are the  $i$ -th predicted and ground-truth AST tokens, respectively.

For unconditional CAD code generation, consistent with existing approaches [Wu et al. 2021; Xu et al. 2023], we adopt MMD, JSD, Coverage (COV), Novelty Score (Novel), and Invalid Rate (IR) to evaluate diversity, realism, and validity. Specifically, COV measures coverage of generated samples over the real data distribution, Novel

Table 3. Ablation study on the choice of base LLM for text-to-CAD generation.  $ACC_{ast}$  and F1 are reported in percentage (%). MCD, MMD, and JSD are scaled by  $10^2$ .

| Method             | $ACC_{ast} \uparrow$ | F1 $\uparrow$ | MCD $\downarrow$ | MMD $\downarrow$ | JSD $\downarrow$ |
|--------------------|----------------------|---------------|------------------|------------------|------------------|
| with Qwen-4B       | 81.74                | 73.91         | 2.98             | 1.19             | 1.26             |
| with Llama-8B      | 85.44                | 78.69         | 2.79             | 1.15             | 1.09             |
| <b>CAD-Factory</b> | <b>82.81</b>         | <b>76.06</b>  | <b>2.90</b>      | <b>1.22</b>      | <b>1.18</b>      |

quantifies the proportion of generated CAD programs unseen during training, and IR reflects the fraction of generated CAD codes that fail to produce valid 3D models.

## D Additional Experiments

### D.1 More Experimental Results

To provide a more comprehensive view of the inference process, Figure 6 presents additional qualitative results of both the inferred CAD AST and the resulting CAD models for text-to-CAD generation under the challenging parameter-free setting. The results demonstrate that CAD-Factory effectively generates CAD models with complex structures (e.g., intricate sketches) and non-trivial topologies (e.g., multiple modeling operations) directly from natural language descriptions, highlighting the robustness and expressive capacity of our system.

Furthermore, Figure 7 illustrates the framework’s capabilities in various downstream CAD applications. Leveraging clear part-level cues from part-specific renderings, our framework supports diverse editing operations, including adding, removing, and modifying parts based on the initial modeling results. In addition, given semantically similar language descriptions, CAD-Factory is able to produce diverse CAD models with variations in fine-grained details such as dimensions, scales, and part proportions. These results further demonstrate the flexibility and practicality of the proposed system for language-driven CAD design.

### D.2 Additional Ablation Study

To evaluate the impact of base LLM selection on overall system performance, we conduct two additional ablation studies: (1) replacing the base model with the comparably sized Qwen-3-4B [Yang et al. 2025], and (2) adopting the stronger Llama-3-8B [Grattafiori et al. 2024] as the backbone for both the planning and coding agents. In both settings, we apply the same CAD AST token initialization, CAD domain-adaptive pretraining, and agentic fine-tuning procedures as in the main experiments. We report  $ACC_{ast}$ , F1, MCD, MMD, and JSD, with results summarized in Table 3. The results indicate that, under comparable model capacity, CAD-Factory is largely model-agnostic to the choice of base LLM, exhibiting only minor fluctuations across metrics while maintaining consistent overall performance. Moreover, upgrading to a stronger base LLM yields moderate improvements but incurs a substantial increase in parameter count and, consequently, higher computational cost.

Table 4. Quantitative results of text-to-CAD generation in the *parameter-partial* setting.  $ACC_{ast}$  and F1 are reported in percentage (%). MCD, MMD, and JSD are scaled by  $10^2$ .

| Method                       | $ACC_{ast} \uparrow$ | F1 $\uparrow$ | MCD $\downarrow$ | MMD $\downarrow$ | JSD $\downarrow$ |
|------------------------------|----------------------|---------------|------------------|------------------|------------------|
| <i>General-Purpose LLMs</i>  |                      |               |                  |                  |                  |
| GPT-4                        | 56.72                | 61.42         | 19.83            | 3.23             | 16.42            |
| Gemini-1.5                   | 62.76                | 59.38         | 23.43            | 2.83             | 15.34            |
| DeepSeek-V3                  | 63.30                | 60.05         | 17.49            | 3.50             | 11.50            |
| Qwen-3-14B                   | 56.76                | 60.15         | 24.96            | 3.04             | 12.45            |
| <i>Coding-Oriented LLM</i>   |                      |               |                  |                  |                  |
| Claude-4                     | 63.11                | 54.92         | 21.02            | 2.97             | 11.58            |
| <i>Multi-modal Methods</i>   |                      |               |                  |                  |                  |
| 3D-PreMise                   | 69.61                | 63.89         | 8.73             | 2.53             | 8.43             |
| CADVerify                    | 68.85                | 61.21         | 9.61             | 2.19             | 8.52             |
| <i>Task-Specific Methods</i> |                      |               |                  |                  |                  |
| Text2CAD                     | 77.82                | 55.99         | 9.91             | 2.24             | 8.17             |
| CADFusion                    | 68.56                | 60.91         | 11.89            | 3.32             | 8.08             |
| <b>CAD-Factory</b>           | <b>86.55</b>         | <b>79.18</b>  | <b>1.59</b>      | <b>1.28</b>      | <b>0.99</b>      |

### D.3 Parameter-partial Text-to-CAD Generation

We further evaluate CAD-Factory in the parameter-partial text-to-CAD generation setting, where input descriptions include intuitively describable parameters (e.g., modeling plane or extrusion distances) while strictly omitting low-level sketch details. This setting is relatively common among users with prior CAD experience and has been explored in several previous studies. Quantitative comparisons with baseline methods are reported in Table 4. Under this setting, CAD-Factory consistently outperforms all baselines, achieving higher structural accuracy ( $ACC_{ast}$ ), which in turn leads to more accurate parameter prediction (F1), improved geometric quality (MCD), and lower MMD and JSD. Qualitative results in Figure 8 further demonstrate that our method effectively captures both geometric descriptions and modeling parameter information conveyed in the text, producing CAD models that are well aligned with the intended design semantics. In contrast, baseline methods often suffer from structural inconsistencies, inaccurate parameterization, and noticeable geometric distortions.

### D.4 Additional Cross-dataset Validation

To further evaluate the generalization capability of our system on out-of-distribution data, we conduct additional cross-dataset experiments for parameter-partial text-to-CAD generation on the Fusion 360 Gallery [Willis et al. 2021]. Following the dataset construction procedure, we derive parameter-partial modeling descriptions for the Fusion 360 Gallery and directly apply the trained models for inference. The input text follows the same format as in previous sections, containing only intuitively describable modeling parameters while strictly excluding sketch-level parameters, thereby jointly evaluating the model’s structural reasoning and parameter understanding capabilities.

Quantitative results are reported in Table 5, with qualitative results shown in Figure 9. As shown, CAD-Factory consistently outperforms all baselines across all metrics. These results indicate

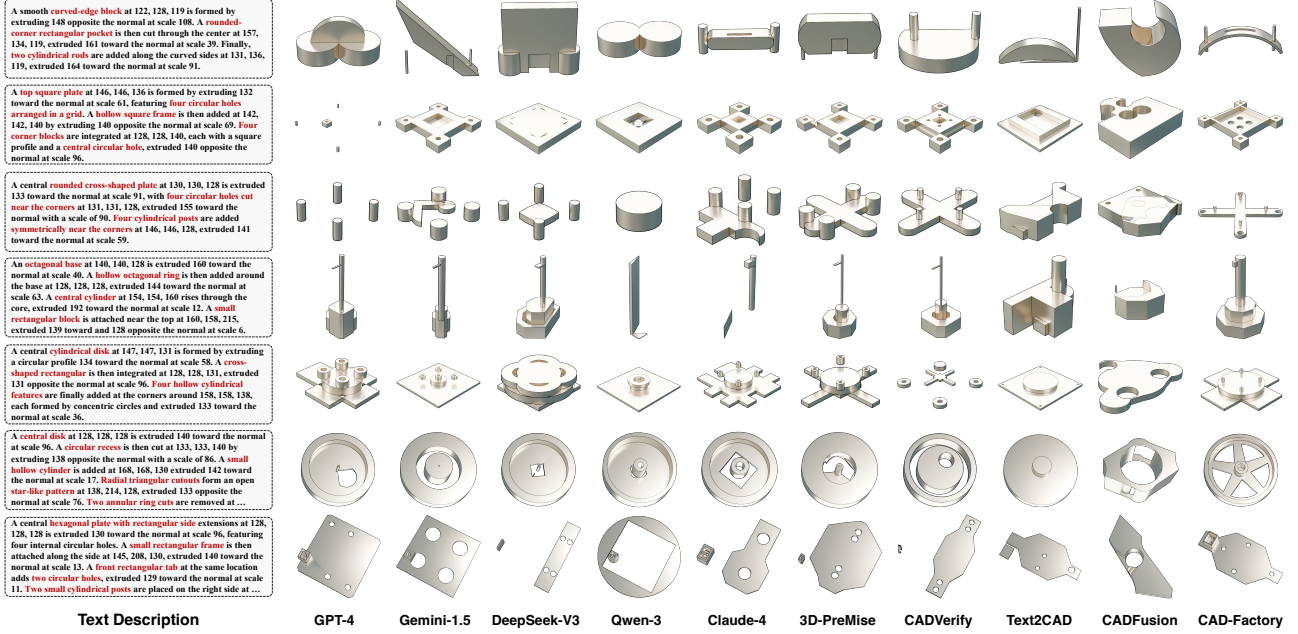
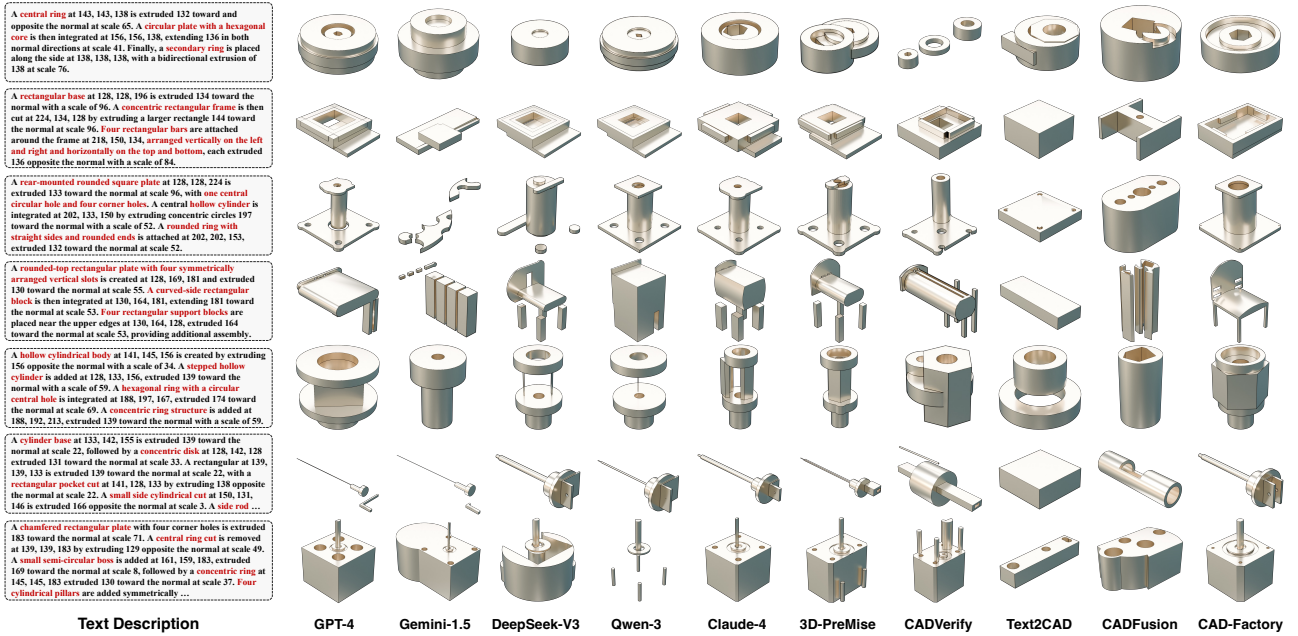




Fig. 7. Additional visualizations of downstream applications. In editing, CAD-Factory uses part-specific renderings with a fixed color palette to establish precise correspondences between instructions and target CAD parts. In diversity modeling, a single language description yields multiple valid models with variations in fine-grained details.

that, even under cross-dataset conditions with partial parameter specifications, our system robustly generates CAD models with structurally coherent assemblies, reasonable parameterization, and

strong geometric consistency. Benefiting from large-scale pretraining, we further observe that LLM- and MLLM-based methods exhibit more stable performance than task-specific approaches across most metrics, consistent with observations in the parameter-free setting.

Fig. 8. Qualitative results of text-to-CAD generation in the *parameter-partial* setting.Fig. 9. Qualitative results of text-to-CAD generation in the *parameter-partial* setting under cross-dataset validation on the Fusion 360 Gallery.

In addition, inspection of the rendered models shows that CAD-Factory maintains high generation stability on out-of-distribution data. In contrast, while baseline methods can often generate individual parts aligned with the input descriptions, they frequently suffer from degraded assembly quality, reduced model plausibility, and geometric inconsistencies.

## E Failure Cases and Discussion

### E.1 Failure Cases

Although CAD-Factory demonstrates strong performance in text-to-CAD generation and supports multiple downstream applications, certain challenges remain. Figure 10 illustrates several representative failure cases in current language-driven CAD generation, all

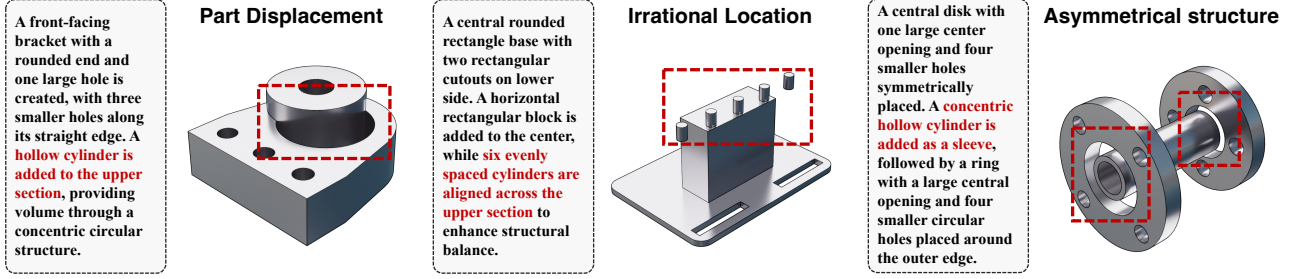


Fig. 10. Failure cases in under-constrained parameter-free text-to-CAD generation. From left to right: (1) part displacement caused by inaccurate parameter prediction, (2) irrational part placement with generated solids located in physically infeasible positions, and (3) asymmetric structures resulting from missing symmetry cues in the input description.

Table 5. Quantitative results of text-to-CAD generation in the *parameter-partial* setting under cross-dataset validation on the Fusion 360 Gallery.  $ACC_{ast}$  and F1 are reported in percentage (%). MCD, MMD, and JSD are scaled by  $10^2$ .

| Method                       | $ACC_{ast} \uparrow$ | F1 $\uparrow$ | MCD $\downarrow$ | MMD $\downarrow$ | JSD $\downarrow$ |
|------------------------------|----------------------|---------------|------------------|------------------|------------------|
| <i>General-Purpose LLMs</i>  |                      |               |                  |                  |                  |
| GPT-4                        | 62.77                | 58.98         | 15.42            | 4.55             | 18.72            |
| Gemini-1.5                   | 55.43                | 48.90         | 27.77            | 5.63             | 21.81            |
| DeepSeek-V3                  | 44.88                | 52.68         | 24.67            | 5.81             | 20.67            |
| Qwen-3-14B                   | 43.89                | 50.53         | 25.38            | 4.70             | 21.94            |
| <i>Coding-Oriented LLM</i>   |                      |               |                  |                  |                  |
| Claude-4                     | 58.57                | 58.72         | 14.61            | 3.90             | 6.37             |
| <i>Multi-modal Methods</i>   |                      |               |                  |                  |                  |
| 3D-PreMise                   | 65.66                | 58.91         | 15.19            | 3.98             | 8.40             |
| CADVerify                    | 65.76                | 57.47         | 17.67            | 4.98             | 11.04            |
| <i>Task-Specific Methods</i> |                      |               |                  |                  |                  |
| Text2CAD                     | 54.11                | 52.87         | 17.42            | 5.59             | 16.72            |
| CADFusion                    | 65.82                | 55.34         | 26.24            | 6.96             | 17.02            |
| <b>CAD-Factory</b>           | <b>78.43</b>         | <b>65.73</b>  | <b>1.93</b>      | <b>2.01</b>      | <b>5.82</b>      |

arising under the parameter-free setting. First, under this under-constrained setting, the absence of modeling parameter cues can lead to inaccurate parameter predictions (e.g., sketch position or orientation), resulting in noticeable part displacement. Second, while shape-correct solids can be easily generated based on input descriptions, the lack of explicit positional information may cause irrational placements or hinder the accurate recognition of underlying symmetrical design intent. These limitations stem from a common cause: ambiguity and missing information in natural language descriptions, which hinder accurate reasoning about geometry and assembly logic.

Beyond addressing these challenges, a promising direction is to incorporate functional modeling intent into text-to-CAD generation. Unlike purely geometric specifications, functional intent captures the purpose, constraints, and expected behavior of a design, allowing the generation process to align more closely with real-world engineering objectives. Integrating such functional semantics into CAD agents would not only improve the usability and practicality of generated models, but also represent an important step toward

autonomous, goal-oriented language-driven CAD generation systems.

## E.2 Discussion

**Intermediate Representations for CAD Generation.** Text-to-CAD generation is inherently under-constrained, as natural language typically specifies high-level design intent while leaving structural organization and geometric details implicit. Unlike standard code generation, parametric CAD programs involve tightly coupled dependencies among part hierarchy, operation ordering, and parameter instantiation, all of which must satisfy execution and geometric constraints. Direct text-to-code generation therefore requires jointly inferring structure and parameters, making the problem particularly challenging.

Intermediate representations provide a principled way to address this challenge by restructuring the generation process. Instead of treating CAD generation as a flat sequence prediction task, an intermediate representation introduces an explicit structural scaffold, allowing the model to first resolve high-level organization before committing to low-level geometric details.

In this work, we adopt an AST as a CAD-specific intermediate representation. The AST captures key modeling structure—such as part decomposition, operation sequencing, and dependency relations—while deferring numerical specification to a later stage. This design does not aim to redefine CAD programs, but to provide a minimal yet structured interface between language understanding and executable modeling instructions. More broadly, this suggests that the choice of intermediate representation is a key factor in adapting large language models to structured design domains, where appropriate abstractions can simplify inference and improve generation reliability.

**System Extensibility.** The current system is developed within the “*sketch-extrusion*” paradigm, a canonical formulation in practical CAD designs. A natural question is whether this formulation can be extended to support more expressive and complex modeling operations. A key limitation lies in the scope of existing data: most open-source datasets are centered on sketch-extrude designs and provide limited coverage of richer operations or intermediate geometry references commonly used in real-world CAD modeling.

Importantly, CAD-Factory is not inherently tied to specific operation types. The AST provides a structural abstraction over modeling primitives, operation sequences, and inter-part dependencies,

independent of particular operations, thereby enabling modular extension to additional operations such as revolve, sweep, fillet, or references to intermediate geometry. At the system level, the structured generation process naturally generalizes to such settings: the planning stage infers high-level organization, while the coding stage instantiates operation-specific parameters under structural constraints.

## References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2024. GPT-4 Technical Report. arXiv:2303.08774
- Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Haider Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. 2025. Generating CAD Code with Vision-Language Models for 3D Designs. In *The Thirteenth International Conference on Learning Representations (ICLR)*. Singapore, 52236–52262.
- Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. arXiv:2308.12966
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations (ICLR)*. Virtual.
- Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. 2024. Text2CAD: Generating Sequential CAD Designs from Beginner-to-Expert Level Text Prompts. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37. Curran Associates, Inc., Vancouver, BC, Canada, 7552–7579. doi:10.52202/079017-0242
- Jiahao Li, Weijian Ma, Xueyang Li, Yunzhong Lou, Guichun Zhou, and Xiangdong Zhou. 2025. CAD-Llama: Leveraging Large Language Models for Computer-Aided Design Parametric 3D Model Generation. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Vancouver, BC, Canada, 18563–18573. doi:10.1109/CVPR52734.2025.01730
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Cheng-gang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437
- AI Meta. 2024. Llama 3.2: Revolutionizing edge ai and vision with open, customizable models. *Meta AI Blog*. Retrieved December 20 (2024), 2024.
- Wamiq Para, Shariq Bhat, Paul Guerrero, Tom Kelly, Niloy Mitra, Leonidas J Guibas, and Peter Wonka. 2021. SketchGen: Generating Constrained CAD Sketches. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. Curran Associates, Inc., Red Hook, NY, USA, 5077–5088.
- Juergen Riegel, Werner Mayer, and Yorik van Havre. 2016. FreeCAD.
- Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv:2403.05530
- Ruiyu Wang, Yu Yuan, Shizhao Sun, and Jiang Bian. 2025. Text-to-CAD Generation Through Infusing Visual Feedback in Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, Vol. 267. PMLR, Vancouver, BC, Canada, 65326–65345.
- Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. 2021. Fusion 360 gallery: a dataset and environment for programmatic CAD construction from human design sequences. *ACM Trans. Graph.* 40, 4, Article 54 (2021), 24 pages.
- Rundi Wu, Chang Xiao, and Changxi Zheng. 2021. DeepCAD: A Deep Generative Network for Computer-Aided Design Models. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Montreal, QC, Canada, 6752–6762. doi:10.1109/ICCV48922.2021.00670
- Xiang Xu, Pradeep Kumar Jayaraman, Joseph George Lambourne, Karl D.D. Willis, and Yasutaka Furukawa. 2023. Hierarchical Neural Coding for Controllable CAD Model Generation. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, Vol. 202. PMLR, Honolulu, HI, USA, 38443–38461.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. arXiv:2505.09388
- Zeqing Yuan, Haoxuan Lan, Qiang Zou, and Junbo Zhao. 2024. 3D-PreMise: Can Large Language Models Generate 3D Shapes with Sharp Features and Parametric Control? arXiv:2401.06437